



JavaScript基础

001.输出

```
document.getElementById("").innerHTML innerText = ""  
document.write() //测试用，文档加载完后使用会清楚html元素  
window.alert() //警示框，可省略windows  
console.log() //浏览器控制台显示
```

002.语句

值、运算符、表达式、关键字、注释、分号

003.语法

命名、解释上述名词

004.注释

```
//  
/**/
```

005.变量

命名

自动声明

var（应被替代）

const（优先使用，类型与值不需改变）

let

006.let（可变变量使用）

let块作用域、不可重复声明、可变、可计算

007.const（默认使用）

const块作用域、不可重复声明、无法重新分配、变量内容与类型不可变、元素与属性可变

008.运算符

算数运算：+、-、*、**、/、%、++、--

赋值运算：=、+=、-=、*=、/=、%=、**=

比较运算：==、===、!=、!==、>、<、>=、<=、?

字符串运算：比较使用字母顺序、+、+=用于连接

逻辑运算：&&、||、! 与或非

按位运算：&、|、~、^、<<、>>、>>>

三元运算

类型运算：typeof、instanceof

009.数据类型

数据类型是动态可变的

字符串、数字、bigint、布尔值、未定义、空值、符号、对象（数组、日期、地图、集合、整数数组、浮点数组、承诺）

typeof可查看数据类型、空值为字符串、未定义为未定义

010.函数

function name（属性,,,）

调用，事件发生、js代码调用、自我调用

返回，return停止运行、计算返回值给调用者

（）访问函数，返回结果；不使用（）访问函数，返回函数本身

不要使用变量存储函数返回值，使用该函数作为变量值

011.对象

创建对象，直接创{123...}，创空的另加属性，不建议使用new

属性{property: "nb"}，方法{property: function () {}}

访问对象属性，object.property、object【"property"】索引也可以

添加属性，person.property = ""

删除属性，delete person.property、【property】

嵌套对象，person.property.property1、person【property】【property1】

访问对象方法，object.method () 返回函数结果，object.method，返回函数定义

添加方法，person.method = function () {}

012.显示对象

按名称显示对象属性、循环显示对象属性、使用Object.values ()、使用JSON.stringify ()

名称显示：person.name + "," + person.age + "," + person.city;

循环显示：let text = "";

```
for (let x in person) {
```

```
text += person[x] + " ";
```

```
};
```

Object.value：const myArray = Object.values(person);

Object.entries：let text = "";

```
for (let [fruit, value] of Object.entries(fruits)) {
```

```
text += fruit + ": " + value + "<br>";
```

```
}
```

JSON.stringify：let mystring = JSON.stringify(person);

013.对象构造函数

使用函数统一制造对象

多用this，对象函数，使用默认属性值

```
function Person(first, last, age, eyecolor) {
```

```
this.firstName = first;
this.lastName = last;
this.age = age;
this.eyeColor = eyecolor;
this.nationality = "English";
}
```

构造函数完成后可使用 `x = new person(属性)` 批量制作新对象

针对新对象添加属性，`x.property = "111"`

为构造函数添加新属性：`person.prototype.property = "..."`

构造函数的方法：`this.fullName = function() {`
`return this.firstName + " " + this.lastName;`
`};`

为构造函数添加新方法：`Person.prototype.changeName = function (name) {`
`this.lastName = name;`
`}`

内置js构造函数，`new Object ()` 使用 `{}` 代替、`Array ()` 使用 `【】` 代替、`Map ()`、`Set ()`、`Data ()`、`RegExp ()` 使用 `/ () /` 代替、`Function ()` 使用 `() {}` 代替

014.JS事件

常见事件：元素改变、点击元素、鼠标悬停、鼠标移开、按键按下、加载

015.字符串修改方法

注意返回值是否需要变量去承接，

字符串长度：`text.length`;

提取字符串索引：`text.charAt (0)`;`text.charCodeAt (0)`;`text.at (2)`;`text`
`【2】`;

提取字符串部分：`text.slice (7, 13)`;`text.substring (7, 13)`;

转换大小写：`text.toUpperCase ()`，`toLowerCase ()`

字符串连接：`text1.concat ("", text2)`，建议加法+

删除字符串两侧空格：text.trim ()、text.trimStart ()、text.trimEnd ()

填充字符串，用0填到长度4：text.padStart (4, "0")、text.padEnd ()

重复字符串4次：text.repeat (4)

替换部分字符串2换1：text.replace (word1, word2)

替换全部字符串：text.replaceAll (text1, text2)

字符串拆分按，号：text.split (",")

016.字符串搜索

返回字符串索引：text.indexOf ("关键词") 未找到返回-1

返回最后一次索引：text.lastIndexOf ("locate", 15)，两种方法均接受第二个参数为起始位置

字符串搜索：text.search ("word") 不可指定起始、可使用正则表达式

字符串匹配：text.match ("word")、text.matchAll ("word") 返回数组，字符串与匹配结果

字符串包含：text.includes ("word")，返回true/false

指定开头结尾：text.startsWith ("word")，text.endsWith ("word")，可加第二参数定起始、终止位置

017.模板字符串

模板字符串使用反引号`而不是引号"" "

模板字符串内允许使用引号

模板字符串允许多行字符串：let text =

`The quick

the lazy dog`;

模板字符串可以将变量与表达式插入字符串\${...}

替换变量：let text = `Welcome ${firstName}, ${lastName}!`;

替换表达式：let total = `Total: ${(price * (1 + VAT)).toFixed(2)}`;

JavaScript中HTML模板：

let header = "Template Strings";

let tags = ["template strings", "javascript", "es6"];

```
let html = `<h2>${header}</h2><ul>`;
for (const x of tags) {
  html += `<li>${x}</li>`;
}
html += `</ul>`;
```

018.数字

数字与字符串相加

在数字运算中，-、*、/，js会把字符串转换为数字

-NaN与Infinity类型为number-

==与===

019.BigInt

BigInt不允许有小数部分

整数末尾或者调用BigInt函数来创建BigInt

```
let y = 99999999999999999n;
```

```
let y = BigInt(1234567890123456789012345)
```

最大与最小安全整数

```
let x = Number.MAX_SAFE_INTEGER;
```

```
let x = Number.MIN_SAFE_INTEGER;
```

判断整数、安全整数的数字方法

```
Number.isInteger(10.5);false
```

```
Number.isSafeInteger(12345678901234567890);false
```

020.数字方法

以不同类型来返回数字

无法转换则返回NaN

对数字变量使用方法：

(x).toString () 以字符串形式返回

x.toExponential (2) 四舍五入以指数表示，返回字符串

x.toFixed (2) 指定小数数位，返回字符串

x.toPrecision () 指定长度数字，返回字符串

x.valueOf () 数字形式返回数字

Number (x / new Date ("1970-01-01")) 将变量或日期转换为数字

parseInt ("10.33") 字符串返回整数，允许使用空格，仅返回第一个数字

parseFloat ("10.33 year") 解析字符串返回数字，仅返回第一个数字

对数字对象对象使用方法：

Number.isInteger(10.5);false判断是否为整数

Number.isSafeInteger(12345678901234567890);false判断是否为安全整数

Number.parseFloat ("10.33 月") 返回第一个数字

Number.parseInt ("10.33") 返回第一个数字

021.数字属性

数字属性不可用于变量，这些属性仅用于查看，类似于固定值，只可以由固定形式访问

Number.EPSILON是大于1的最小浮点数与1之间的差

Number.MAX_VALUE是一个代表js中最大可能数字的常数

Number.MIN_VALUE最小可能数字

Number.MAX_SAFE_INTEGER是js中最大安全整数

Number.MIN_SAFE_INTEGER最小安全整数

Number.POSITIVE_INFINITY正无穷大

Number.NEGATIVE_INFINITY负无穷大

Number.NaN非数字

022.数组

数组可以保存单个名称下的多个值，通过索引访问，建议相同的一个类

数组优势在于内置的属性与方法

声明：const x = 【】

引用或添加或更改：x【0】 = "word"

转化为字符串：x.toString ()

数组的变量性质：访问全部直接引用数组名，

数组的对象性质：typeof返回对象，[object.property](#)访问值慎用，[区分对象与数组，常用位置索引与命名索引区分](#)

数组元素可以是对象、函数、数组

内置属性与方法：x.length、x.sort ()

添加元素：x.push【x.length】 = "word"不要添加过高索引元素

识别数组：Array.isArray (x)、x instanceof Array返回true、false

嵌套数组与对象：对象与数组可以相互嵌套，访问数组内数组使用for in嵌套循环

023.数组方法

x.length，返回数组长度

x.toString () 将数组转化为字符串，逗号分隔

x.at (2)、x【2】 获取x的第三个元素

x.join ("*") 以指定符号将数组元素转化为字符串，类似toString

x.pop () 删除数组最后一个元素，返回删除的元素

x.push ("word") 在数组末尾添加新元素，返回数组长度，

x.shift () 删除第一个元素并将其他元素索引提前，返回移除的值

x.unshift ("word") 在开头添加新元素，返回数组长度

x.concat (y, z...) 合并数组x、y，也可以合并值，返回新数组，

x.copeWithin (2, 0, 2) 将索引元素复制到指定位置，目标位置，开始元素，末尾元素，后两个可选，包左不包右

x.flat () 将多维数组，展平到一维数组，返回新数组

x.latMap () 先映射数组元素，在通过展开数组来返回新数组

x.splice (2, 0, "word")，添加新项目，2添加位置、0删除元素、word添加的新元素，返回一个仅包含删除的项目数组；使用该方法删除元素不会留下空洞。

x.toSpliced (0, 1) 返回一个改变的新数组，原数组不变

x.slice (1, 3) 从数组元素1开始切出并返回剩余部分，包左不包右

024.数组搜索与查找方法

`x.indexOf("word", 1)` 在数组中搜索元素值并返回位置索引，可选参数第一个起始位置，一般+1

`x.lastIndexOf()` 搜索最后一次位置，可选起始位置

`x.includes("word")` 检查数组中是否存在该元素，true

`x.find(function)` 返回第一个通过测试函数的值

`x.findIndex(function)` 返回第一个通过测试函数的值的索引

`x.findLast(条件)` 从数组末尾开始并返回满足条件的值第一个

`x.findLastIndex(条件)` 查找并返回最后满足条件的索引

025.数组的排序方法

`x.sort()` 按照字母顺序对数组进行排序，按照数字排序可选函数筛选

`x.reverse()` 反转数组中元素，可结合sort实现字母降序

`x.toSorted()` 原数组不变，排序后返回新数组

`x.toReverse()` 同理反转后返回新数组

可选函数：`(function(a, b){return a - b})`：比较排序）`(function(){return 0.5 - Math.random()})`：随机排序）（

```
for (let i = points.length - 1; i > 0; i--) {  
  let j = Math.floor(Math.random() * (i+1));  
  let k = points[i];  
  points[i] = points[j];  
  points[j] = k;  
}: 随机洗牌)
```

026.数组迭代

`x.forEach(function)` 为数组中每一个元素调用一次函数

`x.map(function)` 不改变原数组，每个元素执行函数结果创建一个新的数组

`x.flatMap(fuction)` 每个元素执行函数，然后返回展平后的数组

x.filter (function) 返回一个通过函数结果的新数组
x.reduce (function) 从左到右执行函数，并修改原始数组
x.reduceRight (function) 从右到左
x.every (function) 检查是否所有元素都符合函数
x.some (function) 检查是否存在元素符合函数
x.from ("word") 从任何含有长度的对象中返回一个数组
x.keys () 返回一个仅包含数组键索引的对象
x.entries () 返回包含键值对的对象
x.with (2,"word") 更新数组元素，返回新数组，目标位置与内容
...运算符可以将可迭代对象扩展为更多元素

027.数组常量

使用const创建就完了

028.日期对象

创建日期对象，年、月、日、时、分、秒

ISO 2025-03-20

短日期 03/20/2025

长日期 Mar 20 2025

new Date()使用当前日期，创建日期对象

new Date(*date string*)自主编写、创建日期字符串

new Date(*year,month*)自主编写、创建日期

new Date(*year,month,day*)

new Date(*year,month,day,hours*)

new Date(*year,month,day,hours,minutes*)

new Date(*year,month,day,hours,minutes,seconds*)

new Date(*year,month,day,hours,minutes,seconds,ms*)

new Date(*milliseconds*)

data.toString () 显示日期

data.toDateString () 更易读的格式

data.toUTCString () 使用UTC标准转化日期

data.toISOString () 使用ISO标准转化日期

获取日期的方法：

getFullYear()	Get year as a four digit number (yyyy)
getMonth()	Get month as a number (0-11)
getDate()	Get day as a number (1-31)
getDay()	Get weekday as a number (0-6)
getHours()	Get hour (0-23)
getMinutes()	Get minute (0-59)
getSeconds()	Get second (0-59)
getMilliseconds()	Get millisecond (0-999)
getTime()	Get time, 从1970年开始的毫秒

getUTCDate()	getDate()	Returns the UTC date
getUTCFullYear()	getFullYear()	Returns the UTC year
getUTCMonth()	getMonth()	Returns the UTC month
getUTCDay()	getDay()	Returns the UTC day
getUTCHours()	getHours()	Returns the UTC hour
getUTCMinutes()	getMinutes()	Returns the UTC minutes
getUTCSeconds()	getSeconds()	Returns the UTC seconds
getUTCMilliseconds()	getMilliseconds()	Returns the UTC milliseconds

设置日期：

setDate()	Set the day as a number (1-31)
setFullYear()	Set the year (yyyy)
setHours()	Set the hour (0-23)
setMilliseconds()	Set the milliseconds (0-999)
setMinutes()	Set the minutes (0-59)
setMonth()	Set the month (0-11)
setSeconds()	Set the seconds (0-59)
setTime()	Set the time (milliseconds since January 1, 1970)

029.数字对象

数字属性：

Math.E

Math.PI

Math.SQRT2

Math.SQRT1_2

Math.LN2

Math.LN10

Math.LOG2E

Math.LOG10E

数字方法：Math.method (number)

数字转整数：Math.round (x) 最接近的整数、Math.ceil (x) 向上舍入最近整数、Math.floor (x) 向下舍入最近整数、Math.trunc (x) 返回整数部分

Math.sign (x) x为负数、空数、正数返回-1、0、1

Math.pow (x, y) 返回x的y次方

Math.sqrt (x) 返回x的平方根

Math.abs (x) 返回x的绝对值

Math.sin () 返回角度的正弦值cos

Math.max、min () 返回列表中最大与最小值

Math.random () 返回01间的随机数，包左不包右

Math.log (x) 返回x的自然对数logx (y)

030.比较与逻辑运算

true、false

比较运算符：==、===、!=、!==、>、<、≥、≤

逻辑运算符：&&、||、!

条件运算符：variablename = (condition) ? value1: value2

空值合并运算符：result = value1 ?? value2,如果第一个值为空，则选中第二个值

可选连接运算符：x?.name x的属性有name则返回，没有则返回undefined

031.条件语句

- 用于 `if`

指定当指定条件为真时要执行的代码块

- 用于 `else`

指定当相同条件为假时要执行的代码块

- 如果第一个条件为假，则用于 `else if`

指定要测试的新条件

- 用于 `switch`

指定要执行的多个备选代码块

将表达式的值与每个案例的值进行比较。如果匹配，则执行相关的代码块。如果不匹配，则执行默认代码块。

当 JavaScript 遇到 `break` 关键字时，它会跳出 `switch` 块。

关键字 `default` 指定在没有大小写匹配时运行的代码

```
switch(运算) {
```

```
  case x:
```

```
    case y:
```

```
      // code block
```

```
    break;
```

```
    default:
```

```
  }
```

032.循环-for

```
for (expression1; expression2; expression3) {  
  }
```

expression1 (可选)：多个条件由逗号分隔，在循环开始前执行的代码，一般设置变量 `i = 0`

expression2 (可选): 判断条件, true执行循环, false循环结束, 如果省略表达式2, 需要在循环内设置break终止循环。

expression3 (可选): 任何操作, 再循环一次后执行

033.循环-for in

```
for (key in object){  
}
```

for in 循环遍历object对象, 每次迭代返回一个值x, key用于访问属性的值

```
for (variable in array) {  
}
```

for in循环遍历array数组, 每次访问数组的元素, 类似于array.forEach (function), 为每个数组元素调用一次函数function

034.循环-for of

```
for (variable of iterable) {  
}
```

每次迭代, 下一个属性的值会分配给变量,

```
for (let x of array){  
}
```

for of循环遍历数组

```
for (let x of string){  
}
```

for of 循环遍历字符串

```
for (const x of letters) {  
}
```

for of循环遍历集合set元素

```
for (const x of fruits) {  
}
```

for of循环遍历地图map元素

035.循环-while

```
while (condition) {  
}
```

while循环，记得加入结束条件

```
do {  
}while (condition);
```

do-while循环，即使条件为假也会执行一次代码，注意结束条件

036.中断与继续，break continue

break语句用于跳出switch（），跳出全部循环，跳出标签

continue语句用于中断本次循环，执行下一轮循环

标签：label：{statement}

标签名，标签内容可以是任何语句，循环、代码块等，break可以跳出标签

同理可以用标签来标记循环、switch

037.自制迭代器，可以理解为一个记录结果的函数

next（）方法需要返回具有（value：x，done：false）两个属性的对象

value为迭代器返回的值，如果done为true可以省略

done如果迭代器已完成返回true，如果迭代器已生成新值返回false

```
function myNumbers() {let n = 0;  
return {next: function() {n += 10;  
return {value:n, done:false};}  
};  
}
```

```
const n = myNumbers();
```

```
n.next(); // Returns 10
```

```
n.next(); // Returns 20
```

```
n.next(); // Returns 30
```

038.集合

集合类型为对象，内元素为唯一值，元素可以是任何类型

创建空集合 `new Set ()`

使用 `add ()` 为集合添加元素

使用 `for of` 循环 `set` 查看内部元素

039.集合方法

`letters = new Set(["a","b","c"])` 创建新集合，可以初始化，也可以创建空集再加内容

`letters.add ("d")` 添加元素，若添加元素相同，仅保存第一个元素

`for (const x of letters) {`

`text += x}` 循环查看集合内所有元素，查看不会管数据类型，仅查看值

`letters.has ("d")` 查看集合中是否存在该值，返回 `true`

`letters.forEach (function (x) {`

`text += x})` `forEach` 方法为 `set` 中每一个元素调用函数

`letters.values ()` 返回一个包含 `set` 中值的对象，可以使用迭代器查看

`letters.keys ()` 返回值同 `values` 一致，因此 `set` 与 `map` 相互兼容

`letters.entries ()` 返回一个键值对类型的对象，若没有键则返回 `【value, value】`，即值值对

040.地图

地图类型为对象，以键值对形式存储：

object	Map
不可直接迭代	直接可迭代
没有size属性	具有 size 属性
键必须是字符串（或符号）	键可以是任何数据类型
按键顺序不正确	键按插入顺序排列
有默认键	没有默认键

创建并传递数组给新地图new Map (【】)

创建空地图使用Map.set () 添加新元素值、修改现有元素值

使用maps.get ("keys") 方法获取Map中某个键的值

041.地图方法

maps = new Map (【"x", 500】) 将数组传递给函数来创建新地图

maps.get ("x") 获取地图中x键的值

x.set ("y", 200) 向地图中添加或修改元素值

x.size会返回该地图中元素数量

x.dilete ("x") 用于删除地图指定元素，键值对

x.clear () 用于清除map中所有元素

x.has ("x") 用于检查map中是否含有该键，返回true

x.forEach (function) 用于对map中每一个键值对调用函数

x.entries () 返回一个带有【key, values】的对象，可以用于迭代

x.keys () 返回一个带有key的对象，可以用于迭代

x.values () 返回一个带有value的对象，可用于迭代

Map可以将对象作为键

Map.groupBy (x, function) 方法可以根据函数返回的值对map的对象进行分组。理解为map根据函数可以先一步把x分组，并不改变map

Object.groupBy() 将元素分组为 JavaScript 对象。

Map.groupBy() 将元素分组到 Map 对象中。

042.类型typeof

null 是原始值。但是，typeof 返回"对象"。历史bug

NaN（非数字）的数据类型是**数字**！

typeof type;

043.类型转换

- 将字符串转换为数字：Number (x)
- 将数字转换为字符串：String (x) 与toString () 作用相同
- 将日期转换为数字：Number (d) 与d.getTime () 作用相同

- 将数字转换为日期：
- 将布尔值转换为数字：
- 将数字转换为布尔值：
- 自动类型转换：

5 + null // returns 5 because null is converted to 0

"5" + null // returns "5null" because null is converted to "null"

"5" + 2 // returns "52" because 2 is converted to "2"

"5" - 2 // returns 3 because "5" is converted to 5

"5" * "2" // returns 10 because "5" and "2" are converted to 5 and 2

044.解构

解构赋值语法将对象属性转换为变量，也可以转换数组和其他可迭代对象
理解为，由现有的属性等创建新的变量，即将55拆分为5 和 5

```
let {firstName, lastName} = person;
```

```
let [firstName, lastName] = person;
```

解构不会改变原始对象

解构对象：let {firstName, lastName} = person;属性顺序不重要，

对象属性别名：let {lastName : name} = person;

解构字符串：字符相对

```
let name = "W3Schools";
```

```
let [a1, a2, a3, a4, a5] = name;
```

解构数组：let [fruit1, fruit2] = fruits;

跳过数组值：let [fruit1,,,fruit2] = fruits;逗号代表位置字符

数组位置值：let {[0]:fruit1,[1]:fruit2} = fruits;

其余属性：const [a,b, ...rest] = numbers省略号代表余下部分归于rest

解构映射：let text = "";

```
for (const [key, value] of fruits) {
```

```
}
```

交换js变量：[firstName, lastName] = [lastName, firstName];，交换值或者变量名

045.按位运算

二进制

5 & 1	1
5 1	5
~ 5	10
5 << 1	10
5 ^ 1	4
5 >> 1	2
5 >>> 1	2

十进制转化为二进制：

```
function dec2bin(dec){  
return (dec >>> 0).toString(2);  
}
```

二进制转化为十进制：

```
function bin2dec(bin){  
return parseInt(bin, 2).toString(10);  
}
```

046.正则表达式

正则表达式是形成搜索模式的字符序列，通常与两种字符串方法一同使用

/pattern/modifiers，字符与修饰

使用字符串方法，搜索search ()

```
let text = "Visit W3Schools!";
```

```
let n = text.search("W3Schools");单纯使用字符串
```

```
let n = text.search(/w3schools/i);使用正则表达式结合字符串
```

使用字符串方法，替换replace ()

```
let text = "Visit Microsoft!";
```

```
let result = text.replace("Microsoft", "W3Schools");单纯使用字符串
```

`let result = text.replace(/microsoft/i, "W3Schools");`使用正则表达式结合字符串 modifiers 修饰词:

i: 不区分大小写

g: 全局匹配, 在字符串中查找所有匹配项, 而不是在找到第一个匹配项后停止

m: 多行匹配, 在多行字符串中进行匹配, 使 `^` 和 `$` 锚点匹配每一行的开始和结束, 而不仅仅是整个字符串的开始和结束。

s: 单行匹配, 允许点号 (`.`) 匹配换行符 (`\n`)。

y: 粘性匹配, 从字符串上次结束位置开始匹配

pattern 模式

RegExp 是一个具有与定义属性和方法的正则表达式对象

test () 方法1, 返回 true

`/e/.test("The best things in life are free!");`在该句中寻找字符 e

exec () 方法2, 返回找到的文本, 未找到返回 null

`/e/.exec("The best things in life are free!");`

047. 运算符优先级

048. 错误

js 发生错误时会停止代码并抛出错误, 指抛出异常, 异常可以是

JavaScript `String`、`Number`、`Boolean` 或 `Object`: 这个异常指一般具有两个属性的 error 对象, name 与 message

EvalError	eval () 函数出现错误
RangeError	使用了超出和法治范围的数字
ReferenceError	如果使用未声明的变量
SyntaxError	代码的语法出现错误
TypeError	操作数或参数与运算符或者函数所期望的类型不兼容
URIError	在URI函数中使用了非法字符

try 定义待运行的代码

catch 定义一个代码来处理错误

finally 定义一个无论结果如何都运行的代码

throw定义一个自定义错误

try与catch成对出现

```
try {Block of code to try
}
catch(err) {
Block of code to handle errors
}
```

throw允许自建错误，即自己抛出异常，来避免程序异常停止运行，下列为验证错误示例

```
<script>
function myFunction() {
  const message = document.getElementById("p01");
  message.innerHTML = "";
  let x = document.getElementById("demo").value;
  try {
    if(x.trim() == "") throw "empty";
    if(isNaN(x)) throw "not a number";
    x = Number(x);
    if(x < 5) throw "too low";
    if(x > 10) throw "too high";
  }
  catch(err) {
    message.innerHTML = "Input is " + err;
  }
}
</script>
```

现代浏览器通常会结合使用 JavaScript 和内置 HTML 验证，并使用 HTML 属性中定义的预定义验证规则：input的自我验证

```
<input id="demo" type="number" min="5" max="10" step="1">
```

finally允许在try和catch之后执行代码，无论结果如何

```
try {  
  Block of code to try  
}  
catch(  
  err  
) {  
  Block of code to handle errors  
}  
finally {  
  Block of code to be executed regardless of the try / catch result  
}
```

049.作用域

块范围、函数作用域、全局范围

自动全局：如果为未声明的变量赋值，它将自动成为**全局**变量。即使该值是在函数内部分配的。

var定义的全局变量可以使用window调用

050.提升

提升是指js将声明移至顶部的默认行为

var变量在声明之前可以使用，变量可以先使用后声明

`let` 用和定义的变量 `const` 被提升到块的顶部，但尚未**初始化**。

含义：代码块知道该变量，但直到声明它之前不能使用它。

已经初始化的变量，不可被提升，可理解为赋予数值的变量仅供下列代码使用

051.严格模式

“use strict”；指令仅在脚本或者函数开头被识别，这取决于是全局还是局部代码使用严格模式

严格模式要求：

不允许使用未声明的变量、对象

不允许删除变量、对象

不允许删除函数

不允许重复命名参数

不允许八进制数字文字、八进制转义字符

不允许写入只读属性

不允许删除不可删除的属性

eval、arguments不可用作变量

禁止使用with

eval () 内部不可声明变量

关键字this指的是调用该函数的对象

implements、interface、let、package、private、protected、public、static、yield等词语不建议使用作为变量名

052.this关键字

this本身不是变量无法修改值，this关键字引用的是一个对象，根据使用范围不同，引用的对象也不同

方法中的this：在对象方法中使用，this引用的是该对象

单独使用this：严格模式也是引用的是全局对象，也就是window

函数中的this：函数中全局对象默认绑定的是this也就是window，严格模式下不允许默认绑定，this就是undefined

事件处理程序中的this：this指的是接收事件的html元素

对象方法绑定：this指定的是该对象

显式函数绑定：类似于js的call () 与apply () 方法，可以用于以另一个对象的参数来调用一个对象的方法

函数借用：bind () 方法，一个对象可以从另一个对象借用方法

053.箭头函数（建议多用）

箭头函数允许编写更短的函数，

语法：const functionname = (parameters) ⇒ {};

只有一个参数: `const functionname = parameter ⇒ {};`

如果只有一个参数可以省略括号

无参数: `const functionname = () ⇒ {};`

无参数需要使用空括号

如果函数只有一条语句可以省略大括号, 语句的返回值会作为函数值返回, 若果含有多条语句, 需要使用大括号, 并且显式设置`return`返回值

`this`默认使用上下文对象

054.类class

类就是对象的模板, 用于批量制作对象

例子:

```
class Car {  
  constructor(name, year) {  
    this.name = name;  
    this.year = year;  
  }  
}  
  
const myCar1 = new Car("Ford", 2014);  
const myCar2 = new Car("Audi", 2019);
```

构造函数方法:

- 它必须具有确切的名称“构造函数”
- 创建新对象时自动执行
- 用于初始化对象属性

类方法: 创建语法和对象方法相同, 总是添加一个`constructor`的方法, 然后添加任意数量方法, 理解为将用户输入的参数转化为方法使用的变量

可以将参数发给类方法用于调用方法, 参数可选

例子:

```
class Car {  
  constructor(name, year) {
```

```

this.name = name;
this.year = year;
}
age(x) {
return x - this.year;
}
}

const date = new Date();
let year = date.getFullYear();
const myCar = new Car("Ford", 2014);
document.getElementById("demo").innerHTML=
"My car is " + myCar.age(year) + " years old.";

```

055.模块

模块允许将代码分成单独的文件，使得维护代码更加容易，常见是函数、类、对象或任何其它值

模块通过语句import从外部文件导入，同样模块依赖type = "module"放在script标签中：

```

<script type="module">
import message from "./message.js";
</script>

```

导出：具有函数与变量的模块可以存储在任何外部文件中，到处类型包括命名导出与默认导出

命名导出：在js文件中，可以分别导出、可以以一次性导出

```

const name = "Jesse";
const age = 40;
export {name, age};

```

默认导出：一个文件中只能有一个默认导出

```

export default message;

```

导入：可以使用两种方法导入模块，基于命名导出还是默认导出，命名导出使用花括号，默认导出不用

命名导入：import { name, age } from "./person.js";

默认导入：import message from "./message.js";

056.JSON

JSON是一个存储与传输数据的格式

- 数据以名称/值对的形式存在
- 数据以逗号分隔
- 花括号包含对象
- 方括号包含数组

键值对： "firstName":"John"

对象： {"firstName":"John", "lastName":"Doe"}

数组：

```
"employees":[
  {"firstName":"John", "lastName":"Doe"},
  {"firstName":"Anna", "lastName":"Smith"},
  {"firstName":"Peter", "lastName":"Jones"}
]
```

JSON常见用途是从web服务器读取数据，并在网页中显示数据

将JSON文本转换为JS对象

first：创建一个包含JSON语法的JS字符串

```
let text = '{ "employees" : [' +
  '{ "firstName":"John" , "lastName":"Doe" },' +
  '{ "firstName":"Anna" , "lastName":"Smith" },' +
  '{ "firstName":"Peter" , "lastName":"Jones" } ]}';
```

second:使用JS内置函数JSON.parse () 将字符串转换为对象

```
const obj = JSON.parse(text);
```

third：在页面中使用新的JS对象

```
<script>
document.getElementById("demo").innerHTML =
obj.employees[1].firstName + " " + obj.employees[1].lastName;
</script>
```

057.调试

代码调试：编程代码可能包含语法错误或逻辑错误。许多错误都很难诊断。通常，当编程代码包含错误时，什么也不会发生。没有错误消息，您也不会得到任何指示去哪里查找错误。

搜索（并修复）编程代码中的错误称为代码调试。

JavaScript调试器（控制台）：现代浏览器内置了js调试器，使用调试器，您还可以设置断点（可以停止代码执行的地方），并在代码执行时检查变量。

console.log（）方法用于在调试器窗口显示js值

设置断点：在调试器窗口中，您可以在 JavaScript 代码中设置断点。在每个断点处，JavaScript 将停止执行，并让您检查 JavaScript 值。检查值后，您可以恢复代码的执行（通常使用播放按钮）。

debugger关键字：停止JS运行，调用调试函数，与断点具有相同功能

058.约定俗成

编码约定：变量名以字母开头、运算符周围的空格、代码缩进、以分号结束简单语句、

复杂语句规则：

- 将左括号放在第一行的末尾。
- 在左括号前使用一个空格。
- 将结束括号放在新行上，不带前导空格。
- 不要用分号结束复杂的语句。

对象规则：

- 将左括号放在与对象名称同一行。
- 每个属性和其值之间使用冒号加一个空格。
- 使用引号括住字符串值，而不是数字值。

- 不要在最后一个属性值对后添加逗号。
- 将结束括号放在新行上，不带前导空格。
- 始终以分号结束对象定义。

行长<80：请避免行长超过 80 个字符。如果 JavaScript 语句无法放在一行中，则最好的拆分位置是在运算符或逗号之后。

命名约定：HTML与CSS中使用下划线命名first_name、JS中使用驼峰式命名camelCase

使用小写文件名字 + 文件拓展名

避免使用全局变量、顶部声明变量、初始化变量、使用const声明对象、数组、尽量少使用new、少使用==、少使用eval ()、警惕自动类型转换、使用===比较、使用默认参数值、始终使用default结束switch语句、避免将数字、字符串、布尔值用作对象、

059.常见错误

意外使用赋值运算符：let x = 0; if (x == 10)

期望宽松的比较：if (x == y) if (x === y)

加法与链接：5+5 5+"5"

误解浮标：let x = 0.1; let y = 0.2; let z = x + y z不是0.3，应使用乘除法

let z = (x * 10 + y * 10) / 10;

换行破坏JS字符串：

let x = "Hello:

World!";

分号正确使用结束语句：

使用命名索引访问数据：数组使用编号索引、对象使用命名索引

以逗号结束定义：对象和数组定义中的尾随逗号是合法的。

注意Undefined不为空：先测试不为undefined，在测试不为null

if (typeof myObj !== "undefined" && myObj !== null)

060.提高性能

减少循环中的活动：

for (let i = 0; i < arr.length; i++) {不行不行，每次循环会访问长度

let l = arr.length;

for (let i = 0; i < l; i++) {

减少DOM访问：多次访问 DOM 元素，请访问一次，并将其用作局部变量

减少DOM大小：保持DOM中元素较少，可以更快渲染

避免不必要变量：使用整体方法函数，去代替不必要变量承接

延迟JS加载：将JS放在页面主体底部，可以使浏览器首先加载页面

避免使用with关键字，严格模式下不允许使用该关键字

061.保留字

尽量不使用保留字作为变量、标签、函数名等